

★ 1. Cast Implementation

Overview

This document describes the implementation of the cast expression feature in our language. The cast expression allows type conversion between object types, with appropriate runtime checks to ensure type safety.

Implementation Components

1. Syntax and Parser

Added new syntax for cast expressions in the form `(<Symbol><ExpI> .`

Implementation includes:

```
// New AST nodes for cast expressions
| castI(target_type :: Symbol, // In ExpI
        exp :: ExpI)
| castE(target_type :: Symbol, // In Exp
        exp :: Exp)

// Parser implementation in parse, parse_expr, and parse_method_body
| '($ (id :: Identifier))$exp':
    castI(syntax_to_symbol(id),
          parse(exp))
```

Parser handles cast expressions in three contexts:

- Regular expressions (parse)
- Variable-scope expressions (parse_expr)
- Method body expressions (parse_method_body)

2. Type Checking

Type checking for cast expressions ensures:

- Only object types can be cast
- Casting is allowed between related types (subtype or supertype)
- Null can be cast to any object type

Key implementation:

```
| castI(target_type, exp):  
    def exp_type = recur(exp)  
    match exp_type  
    | objT(exp_class):  
        def target_objT = objT(target_type)  
        cond  
        | is_subtype(exp_type, target_objT, t_classes): target_objT  
        | is_subtype(target_objT, exp_type, t_classes): target_objT  
        | ~else: type_error(exp, "cast cannot succeed between  
unrelated types")  
    | nullT():  
        objT(target_type) // null can be cast to any object  
type  
    | ~else:  
        type_error(exp, "can only cast object types")
```

3. Runtime Behavior

Runtime checking ensures:

- Cast target must be an object or null
- Object must be an instance of target type or its subclass
- Null can be cast to any object type

Implementation:

```

| castE(target_type, exp):
    def val = recur(exp)
    match val
    | objV(class_name, fields):
        if check_class_ancestor(class_name, target_type, classes)
            | val
            | error('#interp, "cast failed: " +& class_name +& "
is not a " +& target_type)
    | nullV():
        nullV() // null can be cast to any object type
    | ~else:
        error('#interp, "can only cast objects"

```

Design Decisions and Rationale

1. Handling of Null

- Null can be cast to any object type
- This matches common OOP language behavior
- Maintains type safety while providing flexibility

2. Bidirectional Cast Relations

- Allows casting between types if either is a subtype of the other
- Supports both upcasting and downcasting
- Type checker warns of definitely impossible casts

3. Type Checking vs Runtime Checks

- Type checker prevents obviously wrong casts (like casting integers)
- Runtime checks handle dynamic type verification
- This split provides both static and dynamic safety

4. Error Messages

- Clear distinction between type errors and runtime errors
- Type errors indicate impossible casts
- Runtime errors indicate failed casts that were potentially valid

Testing Strategy

Tests cover all key aspects:

1. Parser:

```
check: parse('(Posn)new Object()')
      ~is castI(#'Posn, newI(#'Object, []))
```

2. Type Checker:

```
check: typecheck(castI(#'Point, nullI()),
                 [point_t_class])
      ~is objT(#'Point)
```

3. Runtime:

```
check: interp(castE(#'AnotherClass, posn27),
              [posn_class, another_class],
              objV(#'Object, []),
              intV(0),
              [])
      ~raises "cast failed: Posn is not a AnotherClass"
```

Soundness Considerations

The implementation maintains type soundness through:

1. Strict type checking preventing impossible casts
2. Runtime verification of object types
3. Consistent handling of null values

4. Proper tracking of class hierarchies

Conclusion

This implementation provides a robust and type-safe casting mechanism that matches common OOP language features while maintaining the safety guarantees of the type system.